Таблицы

Хеш-таблица как движок хранения

- Данные хранятся в файле, в который можно только дописывать в конец (append-only log).
- В оперативной памяти хранится хеш-таблица (словарь): ключ → смещение (offset) в файле данных.



Запись: добавляем пару „ключ-значение" в конец файла и обновляем хеш-таблицу.

Чтение: по ключу находим смещение в хеш-таблице, переходим по нему в файле и читаем значение

Сегменты

- Разбиваем журнал на сегменты фиксированного размера.
- При заполнении сегмента — закрываем его и начинаем новый.
- Запускаем фоновый процесс уплотнения (compaction) и слияния (merging) сегментов.

Уплотнение (compaction)

- Удаление дубликатов ключей.
- Сохранение только самой последней версии значения для каждого ключа.

Слияние (merging)

- Объединение нескольких небольших сегментов в один новый.
- Позволяет уменьшить общее количество сегментов.



Алгоритм чтения

- Поиск начинается в хеш-таблице

самого свежего сегмента.

- Если ключ не найден - проверяется следующий по старшинству сегмент.
- И так далее, пока ключ не будет найден или не будут проверены все сегменты.

Преимущества append-only подхода

- **Высокая производительность записи.** Последовательная запись на диск намного быстрее случайной.
- **Надёжность.** Упрощение конкурентного доступа и восстановления после сбоев - нет риска „полузаписанных" данных.
- **Борьба с фрагментацией.** Процесс слияния сегментов естественным образом переписывает данные в компактном виде.

Ограничения хеш-движка

- **Ключи должны помещаться в ОЗУ.** Хеш-таблица на диске медленная из-за

случайных чтений, дорогого расширения и сложности разрешения коллизий.

- Неэффективные диапазонные запросы.

Нельзя быстро найти все ключи от 00000 до 99999 - нужно обращаться к хеш-таблице для каждого ключа в диапазоне отдельно.

SS-Table (Sorted String Table)

Особенности

- Данные отсортированы по ключу.
- Каждый ключ встречается только один раз (обеспечивается уплотнением).
- Порядок записи значений не важен - приоритет у более новых значений.

Слияние

- Алгоритм аналогичен сортировке слиянием (merge sort)

- Работает даже когда данные не помещаются в оперативной памяти.
- При конфликте ключей берётся значение из самого нового сегмента.

Разреженный индекс

- Не нужно хранить все ключи — только некоторые ориентиры.
- Пример: известны смещения для "handbag" и "handsome" → "handiwork" находится между ними
- Можно быстро просканировать небольшой диапазон

MemTable

- Сбалансированное дерево в оперативной памяти (красно-черное, AVL).
- Данные сохраняются отсортированными по ключу.

- Быстрая вставка в любом порядке.

Надёжность

- Все операции немедленно записываются в журнал (WAL)
- Журнал неупорядочен — используются только для восстановления.
- После записи MemTable в SS-таблицу журнал удаляется.

Алгоритм работы (LSM-tree)

- Запись. Добавляется в Memtable и параллельно пишется в журнал для надёжности
- Чтение. Поиск в MemTable → последнем сегменте → предыдущем → и т.д.
- Фоновые процессы. При превышении лимита MemTable записывается

в SS-таблицу. Регулярное слияние и уплотнение сегментов.

Фильтр Блума

- Эффективная проверка отсутствия ключа
- Избегание ненужных чтений с диска
- Ответ: „возможно есть"/„точно нет".

Write-Ahead Log

- Файл только для добавления (append-only).
- Быстрая последовательная запись.
- Используется для восстановления после сбоев.

Защёлки (latches)

- Облегчённые версии блокировок

- Защита структур данных при одновременном доступе.
- Высокая производительность (быстрее обычных блокировок, но используются для коротких критических секций).

LSM-tree vs B-tree

- LSM-деревья: обычно быстрее при записи.
- B-деревья: обычно быстрее при чтении.

Модель доступа

Дискретная модель доступа (DAC)

Модель управления доступом, при которой владельцы объектов (таблиц, схем, представлений и т.д.) в базе данных самостоятельно определяют, кто и какие действия может выполнять с этими объектами.

Роли

- В PostgreSQL понятие „пользователь" является частным случаем роли. Каждая роль может иметь или не иметь возможность входа в систему (login privilege). Если у роли установлено свойство LOGIN, она может использоваться как учётная запись запись пользователя.
- Роли могут быть членами других ролей. Это позволяет организовывать группы пользователей – например, создать роль developers, которой будут принадлежать все разработчики, и назначить этой роли общие привилегии.

Команды и атрибуты

```
CREATE ROLE name [attr1, attr2, ...];
ALTER ROLE developer WITH PASSWORD 'newsecret';
```

Атрибуты ролей:

- LOGIN - возможность входа в систему.
- SUPERUSER - суперпользователь, обходящий все проверки прав.
- CREATEDB - может создать базы данных
- CREATEROLE - может создать другие роли
- PASSWORD 'secret' - пароль.

Наследование привилегий

- Роль, являющаяся членом роли, по умолчанию автоматически наследует все её привилегии.

```
GRANT role1 TO role2;
REVOKE role1 FROM role2;
```

Привилегии таблиц

- SELECT

- INSERT
- UPDATE
- REFERENCES
- DELETE
- TRUNCATE
- TRIGGER

Привелеги БД

- CREATE
- CONNECT
- TEMPORARY

Привилеги схем

- CREATE
- USAGE

Права доступа к схеме влияют на возможность создания новых объектов, но сами по себе не контролируют доступ к уже существующим объектам. Для этого необходимо назначать привилеги

непосредственно на уровне объектов.